Laboratorio 2

Programmazione di Sistema | A.A. 2025–26 | 1 e 10 Aprile 2026

1 Esercizio 1 – Pangramma

1.1 Descrizione

Un **pangramma** è una frase che contiene tutte le 26 lettere dell'alfabeto inglese, indipendentemente dal fatto che siano maiuscole o minuscole.

Scrivere un programma che:

1. Legga come argomento di command line il nome di un file di testo.
2. Determini se il testo è un pangramma.
3. Conti quante volte ciascuna lettera appare e stampi la statistica.

I caratteri di punteggiatura e gli spazi sono ignorati nel conteggio.

Note

- Tutti i test forniti passano con `cargo test`.
- Il programma non va in panic se il file è mancante o gli argomenti sono assenti.
- L'output è leggibile e coerente con il formato richiesto.

1.2 File di partenza

Viene fornito il file `pangram_empty.rs` da usare come `main.rs` del progetto. Il file contiene:

- Le firme delle funzioni da implementare: `stats` e `is_pangram`.
- La funzione `run_pangram()` da completare con la logica di I/O.
- Una suite di test già scritta nel modulo `#[cfg(test)]`.

Attenzione

Non modificare le firme delle funzioni né i test esistenti. Implementare le funzioni in modo che tutti i test passino con `cargo test`.

1.3 Esempio

```
# Creare un file sentence.txt con il testo da analizzare, poi:
cargo build
cargo run -- sentence.txt
```

Output atteso se l'alfabeto completo è il contenuto del file:

```
"abcdefghijklmnopqrstuvwxyz" is a pangram
a 1
```

```
b 1
c 1
...
z 1
```

1.4 Suggerimenti

- Per leggere gli argomenti: `let args: Vec<String> = env::args().collect();`
- Usare (e capire) `.chars()` per iterare sui caratteri Unicode di una stringa.
- Usare `.to_ascii_lowercase()` per normalizzare maiuscole/minuscole.

1.5 Domande Addizionali

Domanda

Osservare le due forme di iterazione su uno slice `&[u32]`:

```rust
fn f(v: &[u32]) { for el in v { } }     // caso A
fn f(v: &[u32]) { for &el in v { } }    // caso B
```

Che tipo ha `el` nei due casi? Quale operazione viene svolta implicitamente nel caso B?

2 Esercizio 2 – Slugify

2.1 Descrizione

Con il termine **slug** si intende una stringa convertita in versione semplificata, composta solo dai caratteri `[a-z]` `[0-9]` `-` (ovvero numeri, lettere semplici minuscole e trattino) adatta per URL e chiavi di sistema.

Le regole di conversione sono:

1. I caratteri accentati riconosciuti vengono convertiti nell'equivalente non accentato.
2. Tutto viene convertito in minuscolo.
3. Ogni altro carattere non appartenente a `[a-z][0-9]` viene convertito in `-`.
4. Due `-` consecutivi non sono ammessi, solo il primo viene mantenuto.
5. Un `-` finale non è ammesso, a meno che non sia l'unico carattere della stringa.

2.2 Sviluppo

`Passo 1 - Creare il progetto e la funzione slugify`

Creare con cargo un nuovo package chiamato `slugify`. Definire in `main.rs`:

```
fn slugify(s: &str) -> String {
    todo!()
}
```

`Passo 2 - Funzione di conversione conv`

Definire una funzione ausiliaria:

```
fn conv(c: char) -> char {
   todo!()
}
```

`conv` riceve un carattere **già convertito in minuscolo** e restituisce:

- il carattere stesso, se appartiene a `[a-z][0-9]`;
- la lettera non accentata corrispondente, se presente nella tabella;
- `-` in tutti gli altri casi.

> **Nota**
> La funzione `.to_lowercase()` applicata a un `char` restituisce un iteratore anziché un singolo carattere, poiché in alcune lingue una lettera maiuscola può corrispondere a più lettere minuscole.

Usare la tabella di conversione (i caratteri accentati in SUBS_I corrispondono per posizione ai caratteri non accentati in SUBS_O) fornita nel file `table.rs` per "tradurre" i simboli accentati con i non accentati.

Attenzione

Si noti che `SUBS_I` e `SUBS_O` sono `&str` e **non possono essere indicizzate** con `[i]`. Si può assumere che il carattere in posizione i di `SUBS_I` corrisponde al carattere in posizione i di `SUBS_O`.

Domanda

Perché l'indicizzazione non è permessa? Cosa restituisce `"ciao".len()` rispetto a `"ciao".chars().count()`? E con "città"?

Passo 3 - Unit test

Aggiungere in `main.rs` una sezione di test:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_nome() {
        assert_eq!(valore, valore_atteso);
    }
}
```

Definire almeno i seguenti test, da eseguire con `cargo test`:

- `test_conv_accented` – lettera accentata presente in `SUBS_I`
- `test_conv_plain` – lettera non accentata (deve restare invariata)
- `test_conv_invalid` – carattere non ammesso e non in tabella (es. `'!'`)
- `test_conv_unknown_accent` – carattere accentato non in tabella (es. `''`)
- `test_slug_multiword` – stringa con più parole separate da spazio
- `test_slug_accented` – stringa contenente caratteri accentati
- `test_slug_empty` – stringa vuota
- `test_slug_consecutive_spaces` – più spazi consecutivi
- `test_slug_consecutive_invalid` – più caratteri non validi consecutivi
- `test_slug_only_invalid` – stringa composta solo da caratteri non validi
- `test_slug_trailing_space` – spazio finale
- `test_slug_trailing_invalid` – più caratteri non validi consecutivi alla fine

Passo 4 - Interfaccia da command line

Nel `main()`, leggere la stringa da convertire come argomento da command line usando `std::env::args()`, invocare `slugify` e stampare il risultato.

```
// Esempio di invocazione:  cargo run -- "Questo che slug    sara???"
// Output:
slug: questo-che-slug-sara
```

Nota

Il doppio trattino `--` separa i parametri di `cargo` da quelli del programma. Racchiudere la stringa tra doppi apici fa sì che la shell la passi come un unico argomento anziché spezzarla sugli spazi.

3 Esercizio 3 – Battaglia Navale

3.1 Esercizi propedeutici

Prima di affrontare il testo principale, svolgere i seguenti tre esercizi in funzioni dedicate all'interno di un nuovo progetto Rust.

Passo 1 - Lettura e scrittura di file

Leggere un file `test.txt` e salvare il suo contenuto ripetuto 10 volte nello stesso file. Usare `std::fs::read_to_string` e `std::fs::write`. Gestire l'errore se il file o il percorso non esistono, senza panic.

Domanda

Qual è la differenza tra `std::fs::read`, che restituisce `Vec<u8>`, e `read_to_string`? Provare a leggere un file contenente caratteri accentati con `read` e stampare i byte in esadecimale. Cosa si osserva con `"così"` rispetto a `"ciao"`?

Passo 2 - Enum con valori associati

Definire la seguente enum:

```
use std::time::SystemTime;

enum MyError {
    Simple(SystemTime),
    Complex(SystemTime, String),
}
```

Implementare una funzione `print_error(e: MyError)` che stampi il tipo di errore e i valori contenuti, senza usare `{:?}` (formato debug), gestendo esplicitamente i casi con `match`.

Passo 3 - Funzioni che restituiscono Result

Implementare la funzione:

```
pub enum MulErr { Overflow, NegativeNumber }

pub fn mul(a: i32, b: i32) -> Result<u32, MulErr> {
    todo!()
}
```

La funzione deve restituire:

- `Ok(risultato)` se il prodotto è calcolabile e non negativo;
- `Err(MulErr::NegativeNumber)` se uno solo tra `a` e `b` è negativo;
- `Err(MulErr::Overflow)` se il risultato supera il valore massimo di `u32`.

Nota

Usare `checked_mul` per rilevare l'overflow in modo sicuro.

3.2 Esercizio principale

Il programma gestisce la costruzione di uno schema di **battaglia navale 20×20** salvato su file. La costruzione avviene per passi: alla prima invocazione si crea una board vuota; ad ogni invocazione successiva si aggiunge una nave nella posizione indicata e si salva lo schema aggiornato.

Formato del file

Il file è composto da 21 righe:

- **Riga 1:** quattro interi separati da spazio (es. `4 3 2 1`) che indicano quante navi di lunghezza 1, 2, 3 e 4 possono essere aggiunte.
- **Righe 2–21:** 20 righe di 20 caratteri, con `' '` per le caselle vuote e `'B'` per quelle occupate da una nave.

Interfaccia da command line

Creare una nuova board:

```
cargo run -- board.txt new 4,3,2,1
```

Aggiungere una nave:

```
cargo run -- board.txt add_boat V,3,10,10
```

Il terzo argomento specifica direzione orizzontale o verticale (`H` o `V`), lunghezza e posizione di partenza (riga, colonna). Il verso è sempre dall'alto verso il basso e da sinistra a destra. Gli indici partono da 1.

Esempio: `add_boat V,3,10,10` aggiunge una nave verticale di 3 caselle a partire da $(10, 10)$, occupando le caselle $(10, 10)$, $(11, 10)$, $(12, 10)$.

L'operazione `add_boat` deve essere sicura: stampare un messaggio su `stderr` e non modificare il file nei seguenti casi:

- Sono già state aggiunte tutte le navi disponibili di quella lunghezza.
- La nave si sovrappone o tocca (anche in diagonale) una nave già presente.
- La nave esce dai limiti della board.

Struttura del codice

Implementare i metodi seguenti senza modificarne le firme:

```rust
const BSIZE: usize = 20;

pub struct Board {
    boats: [u8; 4],
    data: [[u8; BSIZE]; BSIZE],
}

pub enum Error { Overlap, OutOfBounds, BoatCount }

pub enum Boat { Vertical(usize), Horizontal(usize) }

impl Board {
```

```rust
    /// Crea una board vuota con la disponibilita' di navi specificata.
    pub fn new(boats: &[u8]) -> Board { todo!() }

    /// Crea una Board a partire dal contenuto del file (come stringa).
    pub fn from(s: String) -> Board { todo!() }

    /// Aggiunge la nave; restituisce la nuova Board o un errore.
    pub fn add_boat(self, boat: Boat, pos: (usize, usize))
        -> Result<Board, Error> { todo!() }

    /// Converte la board in una stringa salvabile su file.
    pub fn to_string(&self) -> String { todo!() }
}
```

Domanda

La firma di `from` riceve `String` anziché `&str`. Questo è necessario? Cosa cambierebbe usando `&str`?

Bonus (facoltativo)

- Identificare e scrivere test per la struct `Board`.
- Modificare `add_boat` in modo che aggiorni la `Board` in-place invece di crearne una nuova. Come va cambiata la firma di `self`?